

Fundamentos de Organización de Datos

Guia teoria



Clase 1: Archivos: introducción

Es la unidad básica de almacenamiento, capaz de contener distintos tipos de datos. Poseen un nombre único en el sistema de archivos del dispositivo. También tienen organización jerárquica de directorios o carpetas.

Tipos

- **Registros de longitud fija**
- **Archivos tipados**
 - **Text:** caracteres estructurados en líneas. Lectura/escritura con conversión automática de tipos. Tienen un acceso exclusivamente secuencial y son útiles para importar y exportar datos.
 - **Bloques de bytes (File):** los datos están almacenados en bloques de bytes. Estos bloques pueden representar distintos tipos de datos y se organizan de manera estructurada utilizando metadatos o delimitadores, así como el tipo de dato dentro de cada bloque, esto permite una interpretación más específica de la información almacenada.

Operaciones básicas en pascal

Definición de archivo tipados

- **var** archivo lógico: **file of** tipo_de_dato;
- **type**
 archivo = **file of** tipo_de_dato;
var archivo logico: archivo;

Clase 2: Archivos: Maestro - detalle

- **Archivo maestro:** resume información sobre el dominio de un problema específico
- **Archivo detalle:** contiene movimientos realizados sobre la información contenida en el archivo maestro

Algoritmo de actualización de un archivo maestro con un archivo detalle

Precondiciones

- Existe un archivo maestro
- Existe un único archivo detalle que modifica al maestro
- Cada registro del detalle modifica a un solo registro del maestro que seguro existe
- No todos los registros del maestro son necesariamente modificados
- Cada elemento del maestro que se modifica, es alterado por uno o más elementos del archivo detalle
- Ambos archivos se ordenan por el mismo criterio

Clase 4: Archivos: Corte de control

Es un proceso mediante el cual la información de un archivo se presenta de una forma organizada de acuerdo a la estructura que posee el archivo.

Ejemplo – Formato

Provincia:

Ciudad:

Sucursal:

Vendedor 1 Total \$\$

...

Vendedor N Total \$\$

Total Sucursal: Total \$\$

Sucursal:

Vendedor 1 Total \$\$

...

Vendedor N Total \$\$

Total Sucursal: Total \$\$

Clase 5: Archivos: Merge

Es un proceso mediante el cual se genera un nuevo archivo a partir de otros archivos existentes.

Clase 6: Archivos: Bajas

Es un proceso que permite quitar información de un archivo. Puede llevarse a cabo de dos modos distintos:

- **Baja física:** consiste en borrar efectivamente la información del archivo, recuperando el espacio físico. En todo momento se administra un archivo que ocupa el espacio mínimo necesario, aunque el performance de los algoritmos sea más complejo.

Para hacerlo se debe generar un nuevo archivo con los elementos válidos, es decir, sin copiar los que se desea eliminar. En el caso de archivos sin ordenar, es necesario generar los reacomodamientos que sean necesarios.

- **Baja lógica:** la información es eliminada pero sin recuperar el espacio físico.
 - **Técnicas:**
 - **Recuperar el espacio:** se utiliza el proceso de baja física periódicamente para realizar un proceso de compactado del archivo, es decir, quitar los registros marcados como eliminados.
 - **Reasignar espacio:** recupera el espacio utilizando los lugares indicados como eliminados para el ingreso de nuevos registros al archivo (altas).

Clase 7: Árboles B

Son árboles multicaminos con una construcción especial de árboles que permite mantenerlos balanceados a bajo costo.

Propiedades

- Cada nodo puede contener como máximo M hijos con (M-1) elementos.
- La raíz no posee hijos o tiene al menos 2.
- Todos los nodos salvo la raíz tienen como mínimo $\lceil M/2 \rceil - 1$ elementos y como máximo M-1 elementos.
- Todos los nodos terminales están al mismo nivel.

Usos

- Organizar el archivo como un árbol B
- Organizar el archivo índice como un árbol B

► Archivo de datos como árbol B

```
const M = ... ; {orden del árbol}
type
  Tdato = record
    codigo: longint;
    nombre: string[50];
  end;
  TNodo = record
    cant_datos: integer;
    datos: array[1..M-1] of Tdato;
    hijos: array[1..M] of integer;
  end;
  arbolB = file of TNodo;
var
  archivoDatos: arbolB;
```

► Archivo índice como árbol B

```
const M = ... ; {orden del árbol}
type
  Tdato = record
    codigo: longint;
    nombre: string[50];
  end;
  TNodo = record
    cant_claves: integer;
    claves: array[1..M-1] of longint;
    enlaces: array[1..M-1] of integer;
    hijos: array[1..M] of integer;
  end;
  TArchivoDatos = file of Tdato;
  arbolB = file of TNodo;
var
  archivoDatos: TArchivoDatos;
  archivoIndice: arbolB;
```

Overflow

Se crea un nuevo nodo.

La primera mitad de las claves se mantiene en el nodo con overflow.

La segunda mitad se traslada a un nuevo nodo.

La menor de las claves de la segunda mitad se promociona al nodo padre.

Bajas

- Si la clave a eliminar no está en una hoja, se debe reemplazar por la menor clave del subárbol derecho.
- Si el nodo hoja contiene por lo menos el mínimo número de claves, luego de la baja, no se requiere ninguna acción adicional.
- Caso contrario, se debe tratar el underflow

Underflow

- Primero se intenta redistribuir con un hermano adyacente. Se intenta dejar cada nodo lo más equitativamente cargado posible.
- Si no es posible redistribuir, debe fusionarse con el hermano adyacente.

Políticas para resolución de underflow

Política izquierda: se intenta redistribuir con el hermano adyacente izquierdo, si no es posible, se fusiona con el hermano adyacente izquierdo.

Política derecha: se intenta redistribuir con el hermano adyacente derecho, si no es posible, se fusiona con el hermano adyacente derecho.

Política izquierda o derecha: se intenta redistribuir con el hermano adyacente izquierdo, si no es posible, se intenta con el hermano adyacente derecho, si tampoco es posible, se fusiona con el hermano adyacente izquierdo.

Política derecha o izquierda: se intenta redistribuir con el hermano adyacente derecho, si no es posible, se intenta con el hermano adyacente izquierdo, si tampoco es posible, se fusiona con el hermano adyacente derecho.

Casos especiales: en cualquier política si se tratase de un nodo hoja de un extremo del árbol debe intentarse redistribuir con el hermano adyacente que el mismo posea.

Aclaración

En caso de underflow primero se intenta **siempre** redistribuir si el hermano adyacente se encuentra en condiciones de hacer la redistribución y no se produce underflow en él.

Clase 8: Árboles B+

Son una mejora sobre los árboles B, pues conservan la propiedad de acceso aleatorio rápido y permiten además un recorrido secuencial rápido.

- **Conjunto índice:** proporciona acceso indizado a los registros. Todas las claves se encuentran en las hojas, duplicándose en la raíz y nodos interiores aquellas que resulten necesarias para definir los caminos de búsqueda.
- **Conjunto secuencia:** contiene todos los registros del archivo. Las hojas se vinculan para facilitar el recorrido secuencial rápido. Cuando se lee en orden lógico, lista todos los registros por el orden de la clave.

Búsqueda B+

Similar a la búsqueda en árboles B. El proceso es simple ya que todas las claves se encuentran en las hojas, deberá continuar con la búsqueda hasta el último nivel del árbol.

Inserción B+

- **Overflow:** el nodo afectado se divide en 2, distribuyendo las claves lo más equitativamente posible. Una copia de la clave del medio o de la menor de las claves mayores se promociona al nodo padre.
- Si no está lleno es igual a los árboles B.

Bajas

Es más simple que en árboles B ya que las claves están siempre en las hojas. Se distinguen dos casos:

1. Si al eliminar una clave, la cantidad de claves es $\geq \lceil M/2 \rceil - 1$, entonces termina la operación. Las claves de la raíz o nodos internos no se modifican por más que sean una copia de una clave eliminada.

Underflow

2. Caso contrario al caso anterior, la cantidad de claves es $< \lceil M/2 \rceil - 1$, entonces debe realizarse una redistribución de claves, tanto en el índice como en las páginas. Si no es posible redistribuir, los nodos se fusionan.

Políticas para la resolución de underflow

Las mismas que en árboles B.

Clase 9: Hashing

Dispersión de archivos

Técnica para generar una dirección base única para una clave dada. Convierte la clave en un número aleatorio que luego sirve para determinar donde se almacena la clave. Utiliza una función de dispersión para mapear cada clave con una dirección física de almacenamiento. Utilizada cuando se requiere acceso rápido por clave.

Tipos

- **Direccionamiento estático:** el espacio disponible para dispersar los registros del archivo está fijado previamente.
- **Direccionamiento dinámico:** el espacio disponible aumenta o disminuye en función de las necesidades.

Consideraciones

- Capacidad de almacenamiento de cada dirección.
- Densidad de empaquetamiento.
- Función de hash
- Método de tratamiento de desbordes.

Función de hash: caja negra que a partir de una clave genera la dirección física donde debe almacenarse el registro.

Colisión: situación en la que un registro es asignado, por la función de hash, a una dirección que ya posee uno o más registros.

Desborde: situación en la cual una clave carece de lugar en la dirección asignada por la función de dispersión.

Densidad de empaquetamiento: relación entre el espacio disponible para el archivo de datos y la cantidad de registros que integran el mismo. **DE = número de registros / espacio total**

Pueden ocurrir desbordes aunque la función de hash sea eficiente y la densidad de empaquetamiento sea baja.

Métodos para resolver colisiones con desborde en dispersión estática

- Saturación progresiva
- Saturación progresiva encadenada
- Saturación progresiva con área de desborde por separado
- Dispersión doble